Hi,

Thanks for sharing your requirements.

As per my understanding and based on your given details, I have prepared the following document. Kindly review it and share your feedback so we can ensure it aligns perfectly with your expectations.

1. Project Overview

A multi-brand AI chatbot platform that allows the client to manage independent chatbot instances for each brand from a single admin dashboard. Each brand operates in complete isolation with its own tone, knowledge base, compliance rules, and product data.

> *One platform. Multiple brands. Zero cross-contamination.*

2. Core Modules

2.1 Brand Management & Isolation

Every brand is a fully isolated tenant within the system. No data, tone rules, or product information is shared between brands unless explicitly configured.

Feature	Description
Brand Profiles	Each brand has its own name, logo, theme colors, and metadata
Data Isolation	Separate database namespaces, vector DB collections, and storage buckets per brand
Independent Config	Tone rules, compliance rules, product data, FAQs are all brand-specific
Scalable Onboarding	New brand can be added via admin panel without any code changes
Scalable Multi-Brand	The system supports multiple brands with full isolation. Architecture is designed for initial support of 5–10 brands, extensible for higher scale through infrastructure upgrades.
Zero-Code Brand Setup	Adding a new brand requires no developer involvement. Admin configures brand name, tone, products, compliance rules, and channel integrations from the dashboard.

2.2 AI Conversation Engine

The core AI layer uses Claude API with Retrieval-Augmented Generation (RAG) to deliver brand-aware, knowledge-grounded responses.

Component	Details
LLM Provider	Claude API (Sonnet) — primary conversational model
RAG Pipeline	User query → Vector search (brand namespace) → Context injection → LLM response
Brand System Prompt	Each brand has a unique system prompt defining tone, vocabulary, and personality
Conversation Memory	Session-based context window to maintain multi-turn conversations
Hallucination Control	Responses are strictly grounded to the brand's knowledge base; no fabricated claims

2.3 Modular Knowledge Base

Each brand maintains its own knowledge base containing product data, FAQs, routines, and brand rules. Data is embedded and stored in a vector database for semantic search.

Data Type	Storage & Handling
Product Catalog	PostgreSQL (master record: name, description, ingredients, price, image URL) + Vector DB (text embeddings for semantic search)
Product Images	Stored in S3/Cloud Storage; URL referenced in PostgreSQL; served to users via channel-specific media APIs
FAQs	Stored in PostgreSQL, embedded in Vector DB per brand namespace
Routines/Regimens	Multi-step routine builder; stored as structured JSON in PostgreSQL, embedded text in Vector DB
Brand Rules	Allowed/disallowed phrases, compliance rules stored in PostgreSQL and injected into system prompt
Image Style Profile	Each brand defines its own image-style rules (e.g., soft-luxury, clinical-luxury, K-beauty minimal, botanical, modern-clean)
Product Card Styling	Image presentation style for product cards follows brand image rules (rounded vs. sharp edges, background color, overlay style)
Routine Card Styling	Visual style for routine step cards matches brand aesthetic
UI Element Styling	All chatbot UI elements (buttons, cards, backgrounds) follow the brand's defined visual style

Note: Image-style rules are configurable per brand from the Admin Panel. Each brand's chatbot interface and product visuals must reflect its unique aesthetic identity.

2.4 Tone & Personality Engine

Each brand defines its own communication personality. The tone engine shapes every AI response to match the brand's voice.

Parameter	Description
Vocabulary Rules	Preferred and avoided words/phrases per brand
Emotional Style	Warm, clinical, luxurious, friendly — configurable per brand
Communication Style	Formal vs casual, emoji usage, response length preferences
Greeting & Sign-off	Custom opening and closing messages per brand
Response Length	Each brand defines preferred response length: Short / Medium / Long. AI strictly follows the configured length per brand.
Short Mode	Crisp, elegant, 1–2 sentence replies. Ideal for luxury brands
Medium Mode	Balanced replies with necessary detail. 2–4 sentences.
Long Mode	Detailed, informative replies when the brand requires in-depth responses.

Micro-Tone Rules

Parameter	Description
Softness Level	Configurable softness intensity per brand (gentle, neutral, direct)
Sensory Language	Enable or restrict sensory words (e.g., "silky", "velvety", "luminous") per brand
Emotional Cues	Define emotional undertones: calming, uplifting, nurturing, confident
Restricted Adjectives	List of adjectives that must not be used per brand (e.g., "cheap", "basic", "harsh")
Clinical Language Control	Clinical or medical-sounding terms are blocked by default unless explicitly allowed per brand
Harsh Word Blocking	Words that feel aggressive, blunt, or non-premium are automatically filtered

Note: Micro-tone rules allow each brand to fine-tune the emotional feel of every AI response beyond basic tone settings. These are independently configurable per brand from the Admin Panel.

2.5 Product-Safe & Compliance Logic

A guardrails layer sits between the AI engine and the user, ensuring every response passes compliance checks before delivery.

Rule	Implementation
No Medical Claims	System prompt instructs no diagnosis/treatment claims; post-processing filter catches violations
Blocked Phrases	Admin-defined disallowed phrases are checked against every response before sending
Safe Ingredients	No unsafe usage advice; responses only reference brand-approved ingredient information
Fallback Handling	If AI cannot answer safely, it responds with a predefined safe fallback message
Brand-Specific Fallback Tone	Each brand can define its own fallback tone profile inside the admin panel. The fallback tone activates automatically whenever the AI cannot answer safely or a compliance rule is triggered.
No Over-Explaining	AI must keep responses concise and within the brand's defined response length. No unnecessary elaboration.
No Medical Tone	AI must not sound clinical or diagnostic unless the brand explicitly allows clinical language.
No Aggressive Upselling	AI must not push products aggressively. Recommendations must feel natural, not salesy.
No Unnecessary Details	AI must only share information that directly answers the user's question. No filler content.

Note: Conversation boundaries are configurable per brand from the Admin Panel. These rules ensure the chatbot behaves like a premium brand advisor, not a generic bot.

2.6 Routine & Recommendation Logic

The chatbot can guide users through personalized skincare routines based on their skin type, concerns, and preferences.

Feature	How It Works
Skin Profiling	Guided questions to determine skin type (oily, dry, combination, sensitive) and concerns (acne, aging, hydration)
Multi-Step Routines	Admin-defined step sequences (cleanse → tone → serum → moisturize) mapped to brand products
Adaptive Logic	Recommendations change based on user answers; each brand has its own recommendation rules
Product Mapping	Each routine step maps to specific products from that brand's catalog with images and descriptions

2.7 Multi-Channel Integration

One AI engine serves all channels. The channel layer handles message format conversion and platform-specific API calls.

Channel	Integration Method	Key Notes
Website Chat	Custom React widget + WebSocket	Embeddable via script tag; fully branded per brand
WhatsApp	WhatsApp Business API (Meta Cloud API)	Webhook-based; 24hr reply window; template messages for outbound
Instagram DM	Meta Graph API	Connected via brand's Instagram Business account

2.8 Admin Panel

A web-based dashboard for the client to manage all brands, products, rules, and chatbot configurations without any code.

Screen	Functionality
Dashboard	Overview of all brands, total conversations, active users, channel stats
Brand Manager	Add/edit/delete brands; configure brand name, logo, colors, description
Tone Settings	Define vocabulary, emotional style, communication style, micro-tone rules, response length, greetings per brand
Product Manager	Add/edit products with name, description, ingredients, images, price; auto-embeds to vector DB
FAQ Manager	Add/edit FAQ entries per brand; auto-embeds to vector DB on save
Routine Builder	Create multi-step routines; map products to steps; set skin-type conditions
Compliance Rules	Manage allowed/disallowed phrases, blocked topics, conversation boundaries per brand
Image-Style Rules	Define visual aesthetic per brand for product cards, routine cards, and UI elements
Channel Config	Connect WhatsApp numbers, Instagram pages per brand
Conversation Logs	View chat history per brand/channel; flag and review flagged conversations
Analytics	Message volume, popular questions, response quality metrics, channel breakdown
Tone Override	Admin can override tone and personality settings for any brand at any time

Vocabulary Override	Admin can instantly update preferred/avoided words and phrases per brand
Compliance Override	Admin can modify compliance rules, blocked phrases, and safety settings in real-time
Routine Override	Admin can update, reorder, or replace routine logic and product mappings per brand
Emergency Override	Admin can instantly disable a brand's chatbot or switch it to a safe-mode fallback response

Note: All override controls take effect immediately without requiring any system restart or redeployment. The founder retains full real-time control over every aspect of each brand's chatbot behavior.

3. System Architecture

3.1 Architecture Flow

> *User Message (any channel) → Webhook/API → Channel Router → Brand Identifier → Load Brand Config → Vector DB Search (brand namespace) → Claude API (brand system prompt + context) → Compliance Filter → Response (via same channel API)*

3.2 Data Flow: Product Image Handling

When an admin adds a product: Image uploads to S3 (returns URL), product record saves to PostgreSQL (with image URL), text content chunks and embeds into Vector DB (with product_id reference). When a customer asks about a product: Vector DB returns matching text + product_id, PostgreSQL lookup fetches image URL and full details, AI generates response text, response sent with product image via channel-specific media API.

4. Technology Stack

Layer	Technology
Backend API	Python / FastAPI
AI / LLM	Claude API (Sonnet) via Anthropic SDK
Vector Database	pgvector (PostgreSQL extension) — brand-isolated namespaces
Relational Database	PostgreSQL — brands, products, users, configs, logs
Task Queue	Celery + Redis — async webhook processing, embedding jobs
File Storage	AWS S3 or equivalent — product images, brand assets
Admin Panel	React.js + Tailwind CSS

Chat Widget	React embeddable component + WebSocket
WhatsApp	WhatsApp Business API (Meta Cloud API)
Instagram DM	Meta Graph API
Embeddings	Voyage AI / OpenAI Embeddings API
Hosting	AWS / GCP (Cloud Run / ECS)

5. Admin Panel Scope

Detailed admin panel scope covering Dashboard, Brand Management, Tone & Personality Module, Compliance & Safety Rules, Product & Knowledge Management, Routine & Recommendation Management, and Integration Settings. (Refer to existing SRS Sections 5.1–5.7 for full details.)

6. Chatbot User Side Scope

Chatbot interface elements include: brand-themed chat window, welcome message, user input box, AI-generated message bubbles, and quick reply options. The chatbot handles FAQ responses, product-safe replies, follow-up questions, skincare routine suggestions, and adaptive conversation. (Refer to existing SRS Section 6 for full details.)

7. Chatbot User-Facing Screens

Screen / Flow	Description
Welcome Screen	Brand-specific greeting with logo and intro message; quick-action buttons (Browse Products, Skin Quiz, FAQ)
Chat Interface	Message bubbles, typing indicator, product cards with images, routine step cards
Skin Quiz Flow	Guided questions: skin type, concerns, preferences → personalized routine recommendation
Product Card	Inline card with product image, name, price, short description, and link to purchase
Routine Display	Step-by-step routine (Step 1: Cleanse with X, Step 2: Tone with Y) with product images
Fallback Message	When AI cannot answer safely: brand-specific fallback message in the brand’s fallback tone

8. Platforms Covered

Platform	Type	Built By Us	Phase	Notes
Website Chat	Widget	Yes (full UI)	Phase 1	Embed via script tag
WhatsApp	API Integration	Backend only	Phase 2	UI is WhatsApp app
Instagram DM	API Integration	Backend only	Phase 2	UI is Instagram app
Admin Panel	Web App	Yes (full UI)	Phase 1	React dashboard

9. Client-Side Requirements

The following items are required from the client's side to proceed with development and integration:

Requirement	Details	Phase
Brand Content	Product data (names, descriptions, images, ingredients, prices), FAQs, tone guidelines per brand	Phase 1
Hosting / Domain	Cloud hosting account (AWS/GCP) or we can set up on client's behalf	Phase 1
Anthropic API Key	Required for AI conversation engine. Client must set up Anthropic account with active billing. Usage-based paid API.	Phase 1
Embeddings API Key	Required for vector search (Voyage AI or OpenAI). Usage-based paid API.	Phase 1
Cloud Storage (S3)	Required for storing product images and brand assets. Usage-based paid service.	Phase 1
Meta Business Account	Required for WhatsApp Business API and Instagram DM integration	Phase 2
WhatsApp Business Approval	Phone number per brand; Meta verification; message template approvals	Phase 2
Instagram Business Pages	Admin access to each brand's Instagram Business account	Phase 2

10. Summary

This document outlines the complete technical architecture for a multi-brand AI chatbot platform tailored for the client's skincare brands. The system is designed with brand isolation at its core, ensuring each brand operates independently with its own AI personality, knowledge base, compliance rules, and product catalog.

The platform covers three key layers:

- A founder-level Admin Panel for no-code brand management, including detailed controls for tone, compliance, products, routines, and channel integrations
- A channel-agnostic AI engine powered by Claude API and RAG, delivering brand-accurate, compliant, and personalized responses
- Multi-channel delivery across website, WhatsApp, and Instagram with a consistent user-facing experience

ADDITIONAL SPECIFICATIONS (Sections 11–19)

The following sections have been added based on client review feedback to ensure production-grade reliability, cost control, operational visibility, and clear delivery terms.

11. Performance & Reliability

11.1 Response Time Targets

The AI chatbot targets a response time of 3 to 6 seconds for standard queries. This includes the full pipeline: channel routing, brand config loading, vector search, Claude API call, compliance filtering, and response delivery.

Parameter	Target
Target Response Time	3–6 seconds (90th percentile). Optimization for faster response times will be an ongoing improvement process.
Timeout Threshold	8 seconds — if AI response exceeds this, system delivers the brand's fallback message instead
Retry Logic	1 automatic retry on Claude API failure before triggering fallback
Vector Search	Target < 500ms per query

> *Response time depends on third-party API performance (Claude API, Embeddings API) which is outside developer control. Targets are best-effort, not guaranteed SLAs.*

11.2 Cold Start Optimization

Optimization	Details
Brand Config Preloading	Frequently used brand configurations are preloaded into memory at server startup
Connection Pooling	Database and Redis connections are pooled to avoid cold connection delays
Common Query Caching	Frequently asked questions and their responses are cached to reduce repeated API calls

12. Cost & Usage Control

Control	Implementation
Max Tokens Per Response	Configurable per brand from admin panel (e.g., 500 tokens for short mode, 1000 for detailed)

Rate Limiting	Per-user message limit (e.g., 30 messages per minute) to prevent abuse and spam
Per-Brand API Tracking	Track API consumption (Claude API calls, embedding calls) per brand for cost visibility

> *API billing (Claude, Embeddings) is the client's responsibility. The system provides usage tracking and configurable limits to control costs.*

13. Logging & Observability

The system maintains comprehensive logs for debugging, quality improvement, and operational monitoring.

Log Type	What Is Stored
Conversation Logs	Full user query and AI response per message, stored per brand and per channel
RAG Retrieval Logs	Which documents/chunks were retrieved from vector DB for each query, with similarity scores
Error Logs	API failures (Claude, Embeddings, S3), timeout events, webhook delivery failures
Compliance Logs	Responses that were blocked or replaced by compliance filter, with the reason
Admin Activity Logs	What was changed, when, and by whom (brand config updates, product changes, compliance rule changes)

All logs are accessible from the admin panel. Sensitive user data (if any) is masked in logs.

14. Channel-Specific Behavior

Each channel has different formatting and behavior rules to optimize user experience per platform.

Channel	Response Style	Media Support	Formatting
Website Chat	Rich and detailed (within brand's length setting)	Product cards, routine cards, quick-action buttons	Full HTML, styled per brand
WhatsApp	Concise and conversational	Product images as media messages, interactive buttons	Plain text, minimal formatting
Instagram DM	Short and conversational	Product images as media messages, quick replies	Plain text, emoji-friendly if brand allows

15. Conversation Intelligence

15.1 Session Personalization

Within a single session, the chatbot remembers user inputs and avoids re-asking questions already answered.

Feature	Behavior
Skin Type Memory	Once user provides skin type during quiz, it is stored in session and used for all subsequent recommendations without re-asking
Concern Memory	User's skin concerns are retained within the session for contextual follow-ups
Preference Memory	Preferences like fragrance-free or vegan are retained within the session

15.2 Conversation Control

Rule	Implementation
No Repeat Recommendations	System tracks products already suggested in the current session and avoids recommending the same product again unless user explicitly asks
No Repeat Questions	If user already answered a question (e.g., skin type), chatbot does not ask again in the same session
Progressive Conversation	Chatbot builds on previous answers rather than starting over on each message

15.3 RAG Governance

Strict controls on how the AI uses retrieved knowledge base content.

Control	Implementation
Minimum Similarity Threshold	Vector search results below a configurable similarity score (e.g., 0.7) are discarded and not sent to the AI
No Speculative Answers	If no relevant data is found above the threshold, AI triggers fallback instead of guessing
Source Grounding	AI can only use information from the retrieved brand knowledge base. No external knowledge or fabricated content.

16. Data & Security

16.1 Data Retention & Privacy

Policy	Details
Conversation Data Retention	Chat logs are retained for a configurable period (e.g., 90 days). Older data is automatically purged.
User Data Deletion	Ability to delete specific user conversation data upon request
Data Masking	Sensitive user data (if captured) is masked in logs and analytics
Privacy Compliance	System is designed to support basic GDPR/CCPA compliance requirements

16.2 Backup & Recovery

Component	Backup Strategy
PostgreSQL Database	Automated daily backups with configurable retention period
Vector DB Embeddings	Re-generable from source data in PostgreSQL. Backup of source data is sufficient.
Product Images (S3)	S3 provides built-in redundancy. Cross-region replication available if needed.
Brand Configurations	Stored in PostgreSQL, covered by database backup
Recovery Process	Documented recovery procedure to restore full system from backups within defined timeframe

17. Knowledge Base Update Handling

When brand content is updated through the admin panel, the system automatically keeps the AI's knowledge in sync.

Action	System Behavior
Product Added	Product record saved to PostgreSQL. Image uploaded to S3. Text automatically chunked and embedded into brand's vector DB namespace.
Product Updated	PostgreSQL record updated. Old embeddings deleted from vector DB. New embeddings generated and stored.
Product Deleted	Record removed from PostgreSQL. Image removed from S3. Embeddings removed from vector DB. Chatbot stops recommending this product immediately.
FAQ Added/Updated	FAQ saved to PostgreSQL. Old embedding replaced with new embedding. Chatbot uses updated answer immediately.

FAQ Deleted	FAQ removed from PostgreSQL and vector DB. Chatbot no longer references this FAQ.
Embedding Status	Admin panel shows embedding sync status (pending, completed, failed) for each update.
Embedding Failure	If embedding API fails, data is saved to PostgreSQL. Embedding job is queued for automatic retry. Admin is notified.

> *No stale data is ever served to users. The system ensures AI responses always reflect the latest brand content.*

18. Project Delivery Terms

18.1 Definition of Completion

The project will be considered complete when:

#	Criteria
1	All core modules (AI engine, admin panel, website chat widget) are functional and testable
2	All features listed in this SRS are implemented as specified
3	QA checklist is passed and approved by the client
4	System is deployed in production environment
5	Documentation, credentials, and source code are handed over
6	Any feature not explicitly listed in this SRS is out of scope unless agreed separately

18.2 QA Acceptance Criteria

Before production launch, the system must pass a documented QA checklist covering:

#	Test Area
1	Brand isolation — no cross-brand data leakage
2	Tone accuracy per brand — responses match configured tone and response length
3	Compliance-safe responses — no blocked phrases, no medical claims, no hallucination
4	Product recommendation accuracy — correct products for correct skin types
5	Skin quiz logic — correct routine generated based on user answers
6	Website chat widget — functional, branded, responsive
7	WhatsApp integration — webhook receives and responds correctly (Phase 2)
8	Instagram DM integration — webhook receives and responds correctly (Phase 2)
9	Admin panel — all CRUD operations, tone settings, compliance rules, overrides functional

10	Error handling and fallback behavior — graceful degradation on API failures

Client approval is required before production launch.

18.3 Maintenance & Support Scope

Term	Details
Bug-Fix Period	30 days post-launch bug-fix support included at no additional cost
Response Time	Critical bugs: within 24 hours. Non-critical: within 48–72 hours.
Scope	Bug fixes only. New features, enhancements, or scope changes are quoted separately.
API/Hosting Costs	Paid by the client (Claude API, Embeddings API, AWS/GCP hosting, S3 storage)
Monthly Maintenance	Available as an optional paid engagement if client requires ongoing support beyond the bug-fix period

18.4 Third-Party Dependencies

The system depends on external services. Responsibility is clearly defined:

Service	Client Responsibility	Developer Responsibility
Claude API	Account setup, billing, usage costs	Integration, error handling, fallback on failure
Embeddings API	Account setup, billing	Integration, retry logic on failure
Meta APIs	Business account, verification, phone numbers, page admin access	Webhook implementation, channel routing
AWS / GCP	Cloud account, billing	Deployment, server configuration
S3 Storage	Storage billing	Upload/retrieval implementation

> *Any downtime, rate limiting, or breaking changes from third-party services are outside developer control.*

18.5 Phase Priority

Phase	Scope	Dependency
Phase 1	AI backend, RAG pipeline, admin panel, compliance engine, website chat widget	None — standalone delivery
Phase 2	WhatsApp Business API integration, Instagram DM integration, channel	Requires Phase 1 to be complete

	routing	

Developer must deliver all Phase 1 features before go-live. Phase 2 is delivered as a subsequent milestone.

19. Source Code Delivery & Ownership

Full source code (backend, frontend, admin panel, database schema, deployment scripts, and configuration files) will be delivered to the client upon completion. The client will have full ownership and unrestricted rights to use, modify, deploy, and extend the system.

Final delivery includes:

#	Deliverable
1	Full backend source code (FastAPI)
2	Full frontend source code (React admin panel + chat widget)
3	Database schema and migration scripts
4	API documentation
5	Deployment scripts and environment variable documentation
6	Setup and installation instructions

> *Ready to proceed to detailed timeline and cost estimation upon approval.*